PROJET PÉDAGOGIQUE — APPRENTIE 2E ANNÉE

GenshinWiki

Wiki collaboratif — Application API Rails

Version 1.0 | **Date** Mars 2026 | **Destinataire** Maryam | **Stack** Ruby on Rails 8

TABLE DES MATIÈRES

01. Contexte & Objectifs
02. Description du projet
03. Modèles de données
04. Diagramme UML
05. API REST
06. Validations & Callbacks
07. Phases & Livrables
08. Phase avancée
09. Contraintes techniques
10. Critères d'évaluation
11. Questions de référence

1 Contexte & Objectifs

Contexte

Dans le cadre de ta formation d'apprentie en informatique, ce projet a pour but de te faire pratiquer et consolider les bases du développement web backend à travers un projet concret, progressif et représentatif des applications que tu rencontreras en entreprise.

Le projet se construit en plusieurs phases. Chaque phase introduit de nouveaux concepts, en s'appuyant sur ce qui a été appris précédemment. L'objectif n'est pas seulement que le code fonctionne, mais que tu sois capable d'**expliquer pourquoi** tu as fait tel ou tel choix.

Objectifs pédagogiques

#	COMPÉTENCE
1	Expliquer et appliquer les principes de la programmation orientée objet (héritage, encapsulation, polymorphisme)
2	Concevoir un schéma de base de données avec des relations cohérentes
3	Écrire des tests unitaires et d'intégration
4	Créer une API REST respectant les conventions HTTP
5	Implémenter un système d'authentification et de gestion des droits
6	Structurer son code avec des services et gérer les erreurs proprement
7	Utiliser les outils professionnels (Rubocop, Swagger, Sidekiq)

2 Description du projet

Présentation

GenshinWiki est une application web de type wiki collaboratif autour du jeu Genshin Impact. Elle permet à des utilisateurs de consulter une base de données de personnages, de les ajouter à leur collection personnelle, et de laisser des notes et commentaires.

L'application expose une **API REST** qui pourrait être consommée par un frontend (web, mobile). La gestion des données se fait entièrement côté backend.

Personas

VISITEUR

Peut consulter les personnages et lire les reviews. Aucun compte requis.

UTILISATEUR

Peut créer un compte, gérer sa collection personnelle, poster des reviews.

ADMINISTRATEUR

Peut créer, modifier et supprimer des personnages. Peut modérer les reviews.

3 Modèles de données

Schéma général

```
User ──────────────────────────────────────────────────────┐
 ├── has_one  :profile                                      │
 ├── has_many :reviews                                      │
 └── has_many :collection_entries                           │
                                                            │
Character (STI)                                             │
 ├── PlayableCharacter                                      │
 │    ├── belongs_to :element                               │
 │    ├── belongs_to :weapon_type                           │
 │    ├── has_many :reviews  ◄──────────────────────────────┘
 │    └── has_many :collection_entries
 │
 └── BossCharacter
      └── has_many :boss_drops

Element       (Pyro, Hydro, Cryo, Geo, Anemo, Electro, Dendro)
WeaponType    (Sword, Bow, Catalyst, Claymore, Polearm)
Tag           (meta, support, healer, dps, …)
CharacterTag  (table de jointure Character <-> Tag)
```

Détail des modèles

User

CHAMP	TYPE	CONTRAINTES
`email`	string	unique, obligatoire
`password_digest`	string	obligatoire
`role`	enum	`user`, `moderator`, `admin` — défaut : `user`
`created_at`	datetime	—

Profile

CHAMP	TYPE	CONTRAINTES
`user_id`	integer	FK, unique
`username`	string	unique, obligatoire
`bio`	text	optionnel
`avatar_url`	string	optionnel
`favorite_element`	string	optionnel

Character (classe de base — STI)

CHAMP	TYPE	CONTRAINTES
`type`	string	discriminant STI (`PlayableCharacter`, `BossCharacter`)
`name`	string	unique, obligatoire
`description`	text	obligatoire, min. 20 caractères
`rarity`	integer	entre 1 et 5
`average_rating`	decimal	calculé automatiquement
`region`	string	—

PlayableCharacter (hérite de Character)

CHAMP	TYPE	CONTRAINTES
`element_id`	integer	FK obligatoire
`weapon_type_id`	integer	FK obligatoire
`base_hp`	integer	> 0
`base_attack`	integer	> 0

CHAMP	TYPE	CONTRAINTES
`base_defense`	integer	> 0
`is_limited`	boolean	défaut : false

BossCharacter (hérite de Character)

CHAMP	TYPE	CONTRAINTES
`is_weekly_boss`	boolean	défaut : false
`location`	string	—
`recommended_level`	integer	—

CollectionEntry — table de jointure enrichie (User ↔ PlayableCharacter)

CHAMP	TYPE	CONTRAINTES
`user_id`	integer	FK
`character_id`	integer	FK
`level`	integer	entre 1 et 90
`constellation`	integer	entre 0 et 6
`notes`	text	optionnel
`Unicité`	—	`user_id` + `character_id` unique

Review

CHAMP	TYPE	CONTRAINTES
`user_id`	integer	FK
`character_id`	integer	FK

CHAMP	TYPE	CONTRAINTES
`rating`	integer	entre 1 et 5, obligatoire
`body`	text	10 à 2000 caractères
`contains_spoiler`	boolean	défaut : false
`Unicité`	—	un utilisateur = une review par personnage

4 API REST

Conventions générales

Toutes les routes sont préfixées par `/api/v1/`. Les réponses sont en **JSON**. L'authentification utilise un token Bearer : `Authorization: Bearer <token>`.

Les erreurs retournent un JSON structuré :

```
{
  "error": "Message lisible",
  "details": { "field": ["message d'erreur"] }
}
```

Authentification

MÉTHODE	ROUTE	DESCRIPTION	AUTH
`POST`	/api/v1/auth/register	Créer un compte	Public
`POST`	/api/v1/auth/login	Obtenir un token	Public

MÉTHODE	ROUTE	DESCRIPTION	AUTH
DELETE	/api/v1/auth/logout	Invalider le token	Auth

Personnages

MÉTHODE	ROUTE	DESCRIPTION	AUTH
GET	/api/v1/characters	Liste paginée	Public
GET	/api/v1/characters/:id	Détail d'un personnage	Public
POST	/api/v1/characters	Créer un personnage	Admin
PATCH	/api/v1/characters/:id	Modifier un personnage	Admin
DELETE	/api/v1/characters/:id	Supprimer un personnage	Admin

Reviews

MÉTHODE	ROUTE	DESCRIPTION	AUTH
GET	/api/v1/characters/:id/reviews	Reviews d'un personnage	Public
POST	/api/v1/characters/:id/reviews	Poster une review	Auth
PATCH	/api/v1/reviews/:id	Modifier sa review	Propriétaire
DELETE	/api/v1/reviews/:id	Supprimer une review	Propriétaire Admin

Collection utilisateur

MÉTHODE	ROUTE	DESCRIPTION	AUTH
GET	/api/v1/users/:id/collection	Voir une collection	Public
POST	/api/v1/users/:id/collection	Ajouter un personnage	Propriétaire
PATCH	/api/v1/collection_entries/:id	Mettre à jour (niveau…)	Propriétaire
DELETE	/api/v1/collection_entries/:id	Retirer de la collection	Propriétaire

Filtres disponibles sur GET /api/v1/characters

PARAMÈTRE	TYPE	EXEMPLE	DESCRIPTION
element	string	?element=pyro	Filtrer par élément
weapon_type	string	?weapon_type=sword	Filtrer par arme
min_rarity	integer	?min_rarity=4	Rareté minimale
type	string	?type=playable	playable ou boss
tag	string	?tag=meta	Filtrer par tag
sort	string	?sort=rarity	Trier par champ
order	string	?order=desc	asc ou desc
page	integer	?page=2	Numéro de page (défaut : 1)
per_page	integer	?per_page=20	Items par page (max : 50)

Diagramme UML

Vue d'ensemble des modèles, de leurs attributs, et de leurs relations.

```
classDiagram direction TB class User { +integer id +string email +string password_digest +enum role +datetime created_at } class Profile { +integer id +integer user_id +string username +text bio +string avatar_url +string favorite_element } class Character { <<STI base>> +integer id +string type +string name +text description +integer rarity +decimal average_rating +string region } class PlayableCharacter { +integer element_id +integer weapon_type_id +integer base_hp +integer base_attack +integer base_defense +boolean is_limited } class BossCharacter { +boolean is_weekly_boss +string location +integer recommended_level } class Element { +integer id +string name +string color } class WeaponType { +integer id +string name } class Tag { +integer id +string name } class CharacterTag { +integer character_id +integer tag_id } class CollectionEntry { +integer id +integer user_id +integer character_id +integer level +integer constellation +text notes } class Review { +integer id +integer user_id +integer character_id +integer rating +text body +boolean contains_spoiler } Character <|-- PlayableCharacter : hérite Character <|-- BossCharacter : hérite User "1" --> "1" Profile : has_one User "1" --> "*" Review : has_many User "1" --> "*" CollectionEntry : has_many PlayableCharacter "*" --> "1" Element : belongs_to PlayableCharacter "*" --> "1" WeaponType : belongs_to PlayableCharacter "1" --> "*" Review : has_many PlayableCharacter "1" --> "*" CollectionEntry : has_many PlayableCharacter "1" --> "*" CharacterTag : has_many Tag "1" --> "*" CharacterTag : has_many
```

Légende des relations

Héritage (STI)

`PlayableCharacter` et `BossCharacter` partagent la table `characters`. La colonne `type` détermine la classe instanciée.

has_one / belongs_to

Un `User` a exactement un `Profile`. La clé étrangère `user_id` est sur la table `profiles`.

has_many :through

`Character` → `CharacterTag` → `Tag`. La table de jointure permet d'ajouter des attributs si nécessaire.

Table de jointure enrichie

`CollectionEntry` relie `User` et `Character` avec des données supplémentaires : level, constellation, notes.

5 Validations & Callbacks

Les règles suivantes doivent être implémentées **dans les modèles**, pas dans les contrôleurs.

```
Character
  ✓ name          : obligatoire, unique
  ✓ rarity        : entre 1 et 5
  ✓ description   : obligatoire, minimum 20 caractères

PlayableCharacter
  ✓ base_hp, base_attack, base_defense : obligatoires, > 0
  ✓ element       : obligatoire
```

```
  ✓ weapon_type    : obligatoire

Review
  ✓ rating         : obligatoire, entre 1 et 5
  ✓ body           : entre 10 et 2000 caractères
  ✓ Un utilisateur ne peut poster qu'une seule review par personnage

CollectionEntry
  ✓ level          : entre 1 et 90
  ✓ constellation : entre 0 et 6
  ✓ Un utilisateur ne peut ajouter un personnage qu'une seule fois

User
  ✓ email          : obligatoire, unique, format valide
  ✓ password       : minimum 8 caractères
```

Callbacks à implémenter

```
Review  after_create  → recalculer average_rating sur le Character associé
Review  after_destroy → recalculer average_rating sur le Character associé
User    before_save   → normaliser l'email en minuscules
```

7 Phases & Livrables

Phase 0 — Mise en place & Organisation

Avant de coder

Avant d'écrire la moindre ligne de code, tu dois mettre en place ton environnement de travail et planifier ton travail comme le ferait une vraie équipe de développement.

1. Créer le dépôt GitHub

- ☐ Créer un nouveau dépôt public sur ton compte GitHub nommé `genshin-wiki-api`
- ☐ Initialiser le projet Rails localement : `rails new genshin-wiki-api --api --database=postgresql`

- [ ] Faire le premier commit et le pousser sur GitHub
- [ ] Ajouter un fichier `README.md` décrivant brièvement le projet
- [ ] Configurer le `.gitignore` pour exclure les fichiers sensibles (`.env`, credentials…)

2. Configurer le projet GitHub

- [ ] Activer **GitHub Issues** sur le dépôt
- [ ] Créer un **GitHub Project** (vue Kanban) avec les colonnes : *À faire*, *En cours*, *En review*, *Terminé*
- [ ] Créer les **labels** suivants pour classifier les tickets :
  - [ ] `model` — création ou modification d'un modèle
  - [ ] `migration` — modification de la base de données
  - [ ] `test` — écriture de tests
  - [ ] `fixtures` — données de test
  - [ ] `bug` — correction d'un problème

3. Créer les tickets de la Phase 1

En te basant sur les livrables de la Phase 1 ci-dessous, réfléchis aux tickets à créer et rédige-les toi-même. Chaque ticket doit avoir : un **titre clair**, une **description** expliquant ce qu'il faut faire, les **labels** appropriés, et une **définition of done** (comment savoir que c'est terminé).

4. Workflow Git à respecter

- [ ] Ne jamais committer directement sur `main`
- [ ] Créer une branche par ticket : `feature/nom-du-ticket`
- [ ] Ouvrir une **Pull Request** pour chaque ticket terminé
- [ ] Référencer le ticket dans la PR : *"Closes #1"* dans la description
- [ ] La PR est mergée uniquement après relecture

CRITÈRE DE VALIDATION

Le dépôt est visible sur GitHub, le projet Kanban est créé, tous les tickets de la Phase 1 sont rédigés et visibles dans la colonne "À faire". Tu peux expliquer à quoi sert chacun des tickets.

Phase 1 — Modèles & Base de données

Phase 1

- [ ] Migrations créées et appliquées
- [ ] Modèles avec associations (`belongs_to` , `has_many` , `has_many :throug`
- [ ] Héritage STI fonctionnel (`PlayableCharacter` , `BossCharacter`)
- [ ] Validations implémentées dans les modèles
- [ ] Fixtures créées (10 personnages min., 3 utilisateurs, éléments, types d'armes)
- [ ] Tests unitaires des modèles (validations, associations, callbacks)

CRITÈRE DE VALIDATION

Les tests passent. Tu peux expliquer pourquoi tu as choisi `has_many :through` plutôt que `has_and_belongs_to_many` pour les tags.

Phase 2 — API REST

Phase 2

- [ ] Routes configurées et versionnées
- [ ] CRUD complet sur les personnages
- [ ] CRUD sur les reviews (avec contrôle du propriétaire)
- [ ] Gestion de la collection utilisateur
- [ ] Réponses JSON cohérentes (structure uniforme)

- [ ] Tests d'intégration des endpoints (status codes, contenu JSON)

CRITÈRE DE VALIDATION

Tous les endpoints sont testables via curl ou Postman. Tu peux expliquer la différence entre un `404` et un `422`.

Phase 3 — Filtres, Pagination & Auth

Phase 3

- [ ] Filtres implémentés via scopes ActiveRecord
- [ ] Pagination fonctionnelle avec métadonnées dans la réponse
- [ ] Authentification par token (Devise ou JWT)
- [ ] Gestion des rôles avec CanCanCan
- [ ] Routes protégées selon les permissions

CRITÈRE DE VALIDATION

Un visiteur ne peut pas poster de review. Un utilisateur ne peut pas modifier la review d'un autre. Un admin peut tout faire.

Phase 4 — Services & Gestion des erreurs

Phase 4

- [ ] Logique métier extraite dans des services
- [ ] Erreurs métier personnalisées (`raise` / `rescue`)
- [ ] Contrôleurs allégés (pas de logique métier directe)
- [ ] Les erreurs retournent des réponses JSON structurées

CRITÈRE DE VALIDATION

Tu peux expliquer pourquoi on utilise un service plutôt que mettre la logique dans le contrôleur ou le modèle.

Phase 5 — Qualité & Documentation

Phase avancée

- ☐ Code conforme à Rubocop (zéro offense)
- ☐ API documentée avec Swagger (apipie-rails)
- ☐ Au moins un job asynchrone (Sidekiq)
- ☐ README à jour avec instructions d'installation

CRITÈRE DE VALIDATION

La documentation Swagger permet de tester l'API sans lire le code. `rubocop` ne retourne aucune offense.

8 Phase avancée

Ces sujets sont abordés une fois les phases de base terminées et bien comprises.

A

Gestion des erreurs — raise / rescue

→ Créer ses propres classes d'erreur métier

→ Attraper les exceptions au bon niveau

→ Retourner des réponses JSON structurées

→ Ne jamais laisser une exception non traitée en production

B

Service Objects

→ Extraire la logique complexe des modèles et contrôleurs

→ Un service = une responsabilité

→ Règle : si une action touche plus d'un modèle → service

→ Facilite les tests unitaires de logique métier

C

Jobs & Crons — Sidekiq

→ Pourquoi ne pas bloquer une requête HTTP

→ Différence job one-shot vs cron récurrent

→ File de messages et notion de queue

→ Idempotence d'un job

D

Internationalisation — gem Mobility

→ Traduire `name` et `description` (FR, EN…)

→ Backend JSONB pour les traductions

→ Différence i18n Rails vs traduction de contenu

E

Rubocop & Guidelines

→ Configurer `.rubocop.yml`

→ Corriger les offenses une par une

→ Comprendre les règles : longueur de méthode, style…

→ Intégration dans le workflow Git (pre-commit hook)

F

Documentation Swagger

→ Documenter chaque endpoint avec apipie-rails

→ Décrire les paramètres, codes de retour, exemples

→ Swagger UI : tester l'API visuellement

→ Documentation = partie du code, pas une afterthought

G

Bonus — Cache

→ Mettre en cache les listes peu modifiées

→ `Rails.cache.fetch` avec expiration

→ Invalider le cache après `after_save` / `after_destroy`

→ Comprendre pourquoi l'invalidation est difficile

H

Bonus — Images ActiveStorage

→ Uploader un sprite par personnage

→ Générer des variants (thumbnail, full)

→ Format WebP, compression

→ Pourquoi ne jamais stocker les images en base

9 Contraintes techniques

LANGAGE

Ruby 3.4+

FRAMEWORK

Rails 8

BASE DE DONNÉES

PostgreSQL

TESTS

Minitest

LINTING

Rubocop

DROITS

CanCanCan

AUTH

Devise

PAGINATION

Kaminari

JOBS

Sidekiq

VERSIONNING

Git — 1 branche / phase

10 Critères d'évaluation

⚙️

Fonctionnement

L'API répond correctement. Les cas d'erreur sont gérés. Les règles de droits sont respectées.

🏗️

Qualité du code

Modèles avec validations, contrôleurs légers, services bien découpés, Rubocop sans offense.

🧪

Tests

Modèles testés (validations, callbacks). Endpoints testés (happy path et cas d'erreur). Tests lisibles.

💬

Compréhension

Tu peux expliquer chaque choix, identifier un bug dans ton code, et dire ce que tu ferais différemment.

11 Questions de référence

Ces questions servent de base lors des points d'avancement. L'objectif n'est pas de les avoir toutes en tête dès le début, mais d'y répondre progressivement.

OOP & Modèles

- ? Quelle est la différence entre une classe et une instance ?
- ? Pourquoi `PlayableCharacter` hérite de `Character` et pas l'inverse ?
- ? Que se passe-t-il si tu fais `Character.all` ?
- ? Quand utilise-t-on `belongs_to` vs `has_many` ?
- ? Différence entre `has_many :through` et `has_and_belongs_to_many` ?

API & HTTP

- ? Différence entre `POST` et `PATCH` ?
- ? Quand retourner un `404` ? Un `422` ? Un `403` ?
- ? Pourquoi versionner une API (`/api/v1/`) ?
- ? À quoi sert le header `Authorization` ?

Tests

- ? Pourquoi écrire des tests si le code fonctionne ?
- ? Différence test unitaire vs test d'intégration ?
- ? Que tester en priorité sur un modèle ?
- ? Pourquoi les fixtures sont-elles importantes ?

Architecture

? Pourquoi déplacer la logique dans un service ?

? Que se passe-t-il si on ne gère pas les exceptions ?

? Pourquoi utiliser un job asynchrone ?

? Quelle est la différence entre auth et autorisation ?

Bonne chance ! N'hésite pas à poser des questions dès que tu bloques — bloquer et chercher fait partie de l'apprentissage. Rester bloquée sans demander d'aide ne sert à rien.